Guía de expositores

APE 4 — Modelo SIR en Python/SimPy y NetLogo

6:00 · 7 diapositivas · 1 expositores

Alejandro · 7 diapositivas · 6:00 1–7

APE 4 — Modelo SIR en Python/SimPy y NetLogo

01
0:00–0:20 · 0:20

Alejandro
cover

Modelo SIR: Python/SimPy vs NetLogo

Hola, soy Alejandro Padilla. En esta APE 4 de Simulación implementé el mismo modelo epidemiológico SIR en dos herramientas, Python con SimPy y NetLogo, y comparé sus resultados para ver en qué se parecen, en qué se diferencian y por qué.

02
0:20–1:10 · 0:50

Alejandro
formula-hero

S → I → R con β y γ

El modelo SIR divide a la población en tres estados: susceptibles, infectados y recuperados. Trabajo con 100 personas, 5 infectados al inicio, durante 100 pasos. En cada paso, cada infectado usa tres números aleatorios: uno elige a quién contacta, al azar entre las otras 99 personas; otro decide si lo contagia, si sale menor que beta, 0.30; y otro decide si se recupera, si sale menor que gamma, 0.10. La razón entre beta y gamma da R0 igual a 3: en promedio, cada infectado contagia a tres personas. En las dos herramientas uso la semilla 42, que fija la secuencia de números aleatorios para que la corrida se pueda repetir.

DIAPOSITIVA 3

APE 4 — Modelo SIR en Python/SimPy y NetLogo

03

1:10–1:50 · 0:40

Alejandro

flow-steps

Python genera, NetLogo muestra

Python calcula la simulación y guarda un CSV con cuatro columnas: tiempo, susceptibles, infectados y recuperados. NetLogo corre su propia simulación con la misma regla y además importa ese CSV para graficarlo. El CSV es el contrato entre las dos: NetLogo no conoce el código de Python, solo el formato del archivo. En Python organicé el código con MVC: el modelo calcula, la vista dibuja y el controlador une las dos partes.

APE 4 — Modelo SIR en Python/SimPy y NetLogo

04

1:50–3:00 · 1:10

Alejandro

live-demo

Demo Python: correr y probar

Muestro la función step de sir_model.py: para cada infectado, elige el contacto al azar, prueba el contagio con beta y la recuperación con gamma. La actualización es síncrona: leo el estado actual y escribo los cambios en una copia. Ejecuto python -m simulacion_ape: con la semilla 42, el pico es de 36 infectados en el paso 21 y al final hay 93 recuperados; también corre 30 réplicas con otras semillas. Abro el CSV, con 101 filas del tiempo 0 al 100, y la curva: los susceptibles bajan, los infectados suben hasta el pico y caen, y los recuperados crecen. Por último, pytest: las 9 pruebas pasan sin dibujar nada, gracias a la separación MVC.

APE 4 — Modelo SIR en Python/SimPy y NetLogo

05

3:00–4:20 · 1:20

Alejandro

live-demo

Demo NetLogo: personas que contagian

Abro NetLogo. Presiono SETUP: cada persona es un agente; azul es susceptible, rojo infectado y gris recuperado. Presiono GO: las personas caminan por la plaza y cada línea roja es un contagio, de quien contagia a quien se infecta. Las líneas son largas porque el modelo usa mezcla homogénea: cualquiera puede contactar a cualquiera, sin importar dónde esté. Cerca del pico la pantalla se llena de líneas; después quedan pocos susceptibles y todo se vuelve gris. La animación es solo visual: el resultado es idéntico con o sin ella. Con la semilla 42, el pico en NetLogo es de 31 infectados en el paso 17. Por último, CARGAR PYTHON importa el CSV de Python y muestra sus curvas junto a las de NetLogo.

APE 4 — Modelo SIR en Python/SimPy y NetLogo

06

4:20–5:10 · 0:50

Alejandro

vs-pair

Una corrida no alcanza: 30 por herramienta

Con la misma semilla, Python da 36 y NetLogo, 31. Cada herramienta tiene su propio generador de números aleatorios, PCG64 en Python y Mersenne Twister en NetLogo, y además consumen los números en otro orden; así cambia quién contagia a quién y la epidemia sigue otro camino. Por eso corrí 30 simulaciones en cada herramienta. Cada punto es una corrida, y el pico va de 24 a 52 infectados. Pero el promedio de Python es 36.1 y el de NetLogo, 35.3, con el pico cerca del paso 19, y ambos están cerca de las ecuaciones, que dan 33. Las dos herramientas simulan el mismo modelo: la diferencia con una sola semilla es solo azar.

APE 4 — Modelo SIR en Python/SimPy y NetLogo

07

5:10–6:00 · 0:50

Alejandro

recap

Lo que aprendí

Me llevo tres cosas. Primero, que una simulación aleatoria no se compara con una sola corrida, sino con muchas réplicas. Segundo, que cada herramienta tiene su fuerte: Python para calcular, probar y analizar datos, y NetLogo para ver el comportamiento de los agentes. Y tercero, que elegir depende del objetivo: para analizar, Python; para mostrar o explicar, NetLogo. Gracias.